

TP N° 4

Dataframes et fichiers CSV

But du TP :

- Manipuler des dataframes.
- Importer/exporter des données depuis/vers des fichiers csv.

Énoncé :

A. Dataframes

Les dataframes sont, comme les matrices, des structures de données constituées de lignes et colonnes. Elles sont utilisées pour stocker des tables de données telles que celles utilisées par les tableurs. Par la suite, nous les utiliserons pour stocker les info des datasets pour entraîner et tester les modèles ML.

Exemple : On veut créer un(e) dataframe pour stocker les informations de la table suivante :

Type de marchandise	disponibilité (tonne)	Volume (m2)	Profit (par tonne)	Pays d origine
semoule		EE	2EEE	rance
ri	E	EE	2 EE	Inde
farine		2EE	EEE	rance
lentille	2E	EE	EE	ésil

Pour créer un dataframe on utilise la fonction **data.frame()** qui prend comme arguments:

- les vecteurs définissant les colonnes avec possibilité de leur donner des noms.
- l'argument **row.names** qui permet de spécifier les noms des lignes. et argument est optionnel mais très recommandé pour simplifier la manipulation par ligne.

Le code qui suit permet de le faire (on a choisit d'utiliser le type de marchandise comme nom de ligne) :

```
myDF<-data.frame(dispo=c(18, 10, 5, 20),  
                 volume=c(400, 300, 200, 500),  
                 profit=c(2000, 2500, 5000, 3500),  
                 origine=c("France","Inde","France","Bresil"),  
                 row.names=c("semoule","riz","farine","lentille"))
```

Q1 : éterminer dans ce code la fa on par laquelle on associe des noms aux colonnes.

Q2 : u est ce que vous remarque dans la one environnement



liquer sur le cercle bleu pour afficher les détails du dataframe **myDF**.

Q3 : comment appelle t il les lignes et colonnes des dataframes Vous pouvez réexécuter le même code en commentant une des colonnes.

Q4 : quelle est le type de la colonne origine correspond il a la précédente capture

considérons le code de création du dataframe **myDF**. Modifier le en ajoutant le paramètre : **stringsAsFactors=TRUE**.

Q5 : Noter les modifications survenues dans **myDF** dans la console environnement puis déduire le rôle de cet argument et sa valeur par défaut.

data.frame (base) R Documentation

Data Frames

Description

The function `data.frame()` creates a data frame, a highly organized collection of variables which share many of the properties characterised by the vector, but which are more flexible and more powerful than any other software.

Usage

```
data.frame(..., row.names = NULL, check.rows = FALSE,
            check.names = TRUE, fix.empty.names = TRUE,
            stringsAsFactors = FALSE)
```

Arguments

... these arguments are of either the form `value` or `tag = value`. Component names are created based on the tag (if present) or the departed argument itself.

row.names NULL or a vector of type character. If non-NULL, it must be the same length as the number of columns, or a character string giving the name of a variable in the data frame.

check.rows If TRUE, the rows are checked for missing values in the frame names.

check.names logical. If TRUE, the names are checked for duplicates and if necessary they are adjusted (by appending a suffix) to make them unique.

fix.empty.names logical. If TRUE, empty names are converted to the value of the first non-empty name. If FALSE, empty names are converted to the value of the first non-empty name. If TRUE, empty names are converted to the value of the first non-empty name.

stringsAsFactors logical. If TRUE, character vectors are converted to factors. If FALSE, they are not. If NULL, the default is TRUE.

omme pour les matrices, les noms de lignes et de colonnes peuvent être consultés ou modifiés après la création du dataframe à l'aide des fonctions **rownames()** et **colnames()**.

Q6 : Exécuter le code suivant, puis noter les modifications survenues dans **myDF**.

```
rownames(myDF)
colnames(myDF)
colnames(myDF)<-c("col1","col2","col3","col4")
```

colnames(myDF)

estituer les noms de colonnes.

Les dataframes peuvent être indexés comme des matrices. En particulier, un élément situé à l'intersection de la ligne <ligne> et la colonne <col> est accessible par <nomDataFrame>[<ligne>,<col>] où <ligne> et <col> sont soit des indices entiers ou les noms de la ligne et de la colonne respectivement.

Q7 : Afficher les valeurs de **myDF[1,2]** et **myDF["semoule","volume"]** puis noter si le résultat est le même.

```
> myDF[1,2]==myDF["semoule","volume"]
[1] TRUE
```

Q8 : Afficher les valeurs de **myDF[,4]** et **myDF[, "origine"]** puis noter si le résultat est le même.

Q9 : Vérifier si l'écriture **myDF[2,"dispo"]** est valable. Quelle est sa valeur

```
> myDF[2,"dispo"]
[1] 10
```

Pour accéder aux données d'un data.frame il y a d'autres moyens comme :

. Il est possible d'extraire une colonne <col> avec la syntaxe <nomDataFrame>\$<col>.

Q10 : Afficher les valeurs de **myDF\$origine** puis noter le résultat.

Q11 : Afficher les valeurs de **myDF\$4** et **myDF\$"origine"** puis noter si le résultat.

2. Il est possible aussi d'utiliser la fonction **attach(<nomDataFrame>)** qui permet de manipuler chaque colonne par son nom comme s'il s'agissait d'une variable.

Q12 : Exécuter les commandes suivantes

```
> dispo <- 123 ; attach(myDF)
The following object is masked by_ .GlobalEnv:
    dispo

> dispo
[1] 123
> volume
[1] 400 300 200 500
> detach(myDF)
> volume
Error: object 'volume' not found
```

Q13 : Qu'est-ce que vous remarquez

attach (base)	R Documentation	Details
Attach Set of R Objects to Search Path		
Description		
The database is added to the R search path. This means that the database is accessed by R when evaluating a variable, an object in the database can be accessed by simply giving their names.		
Details		
By attaching a data frame (or list) to the search path, it is possible to refer to the variables in the data frame by their column names, rather than as components of the data frame (e.g., in the examples below, the first output is easier to write).		
Good practice		
At least two disadvantages of altering the search path and this can easily lead to the wrong object of a particular name being found. People do often forget to detach databases.		

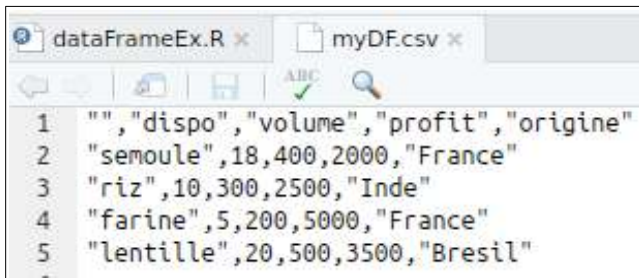
comme règle générale, il est préférable d'éviter l'accès par **attach()** pour éviter les conflits.

B. Sauvegarder des dataframes dans des fichiers csv

Le format CSV (comma separated values) est un format de fichier texte représentant des données tabulées sous forme de valeurs séparées par des virgules. L'utiliser sous R permet de réutiliser ses propres données pour les utiliser entre sessions ou bien de les partager avec d'autres utilisateurs utilisant R ou des tableurs. Les fonctions peuvent être utilisées : `write.csv()` et `read.csv()`.

Q14 : Exécuter la commande `write.csv(myDF, file = "myDF.csv")` puis vérifier si le fichier csv a été créé.

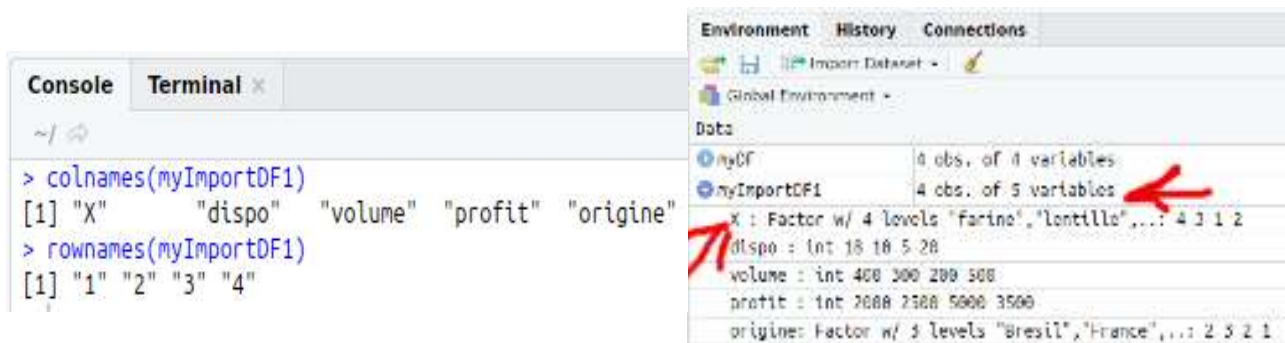
Q15 : Afficher son contenu en utilisant l'explorateur de fichier intégré à R studio, puis en utilisant celui de votre système d'exploitation



Pour importer les données dans un dataframe identique à celui de l'export, quelques précautions doivent être prises. Les étapes suivantes vont nous permettre de le faire :

Q16 : Exécuter la commande suivante puis identifier le problème.

`myImportDF1<-read.csv(file = "myDF.csv")`



Q17 : Afficher les noms de lignes et colonnes pour voir plus clair :

`colnames(myImportDF1)`
`rownames(myImportDF1)`

Q18 : Exécuter le code suivant puis identifier la signification (utiliser pour cela le help) de l'argument ayant permis l'association des noms à leurs lignes.

`myImportDF2<-read.csv(file = "myDF.csv",row.names=1)`
`colnames(myImportDF2)`
`rownames(myImportDF2)`
`str(myImportDF2$dispo)`

```

Console Terminal x
~ / 
> colnames(myImportDF2)
[1] "dispo" "volume" "profit" "origine"
> rownames(myImportDF2)
[1] "semoule" "riz" "farine" "lentille"
> str(myImportDF2$dispo)
int [1:4] 18 10 5 20

```

Q19 : Qu'est-ce que vous remarquez en exécutant `str(myImportDF2$dispo)` ?

Exécutez maintenant le code suivant :

```

myImportDF3<-read.csv(file = "myDF.csv",row.names=1,
                      colClasses = c("character","numeric","numeric",
                                     "numeric","factor"))

colnames(myImportDF3)
rownames(myImportDF3)
str(myImportDF3$dispo)

```

Q20 : Est-ce que la variable `myImportDF3` est identique à `myDF` ?

```

> myImportDF3<-read.csv(file = "myDF.csv",row.names=1,
+                       colClasses = c("character","numeric","numeric",
+                                     "numeric","factor"))
> colnames(myImportDF3)
[1] "dispo" "volume" "profit" "origine"
> rownames(myImportDF3)
[1] "semoule" "riz" "farine" "lentille"
> str(myImportDF3$dispo)
num [1:4] 18 10 5 20

```

myDF	4 obs. of 4 variables
dispo	: num 18 10 5 20
volume	: num 400 300 200 500
profit	: num 2000 2500 5000 3500
origine	: Factor w/ 3 levels "Bresil","France",...: 2 3 2 1
myImportDF1	4 obs. of 5 variables
myImportDF2	4 obs. of 4 variables
myImportDF3	4 obs. of 4 variables
dispo	: num 18 10 5 20
volume	: num 400 300 200 500
profit	: num 2000 2500 5000 3500
origine	: Factor w/ 3 levels "Bresil","France",...: 2 3 2 1

Notons que cette dernière démarche spécifiant les classes des colonnes permet aussi d'accélérer l'importation de manière significative lorsque la taille du fichier csv est importante. Donc il est recommandé de l'utiliser même si la classe par défaut supposée par l'importation est la bonne.